ALLART Noah
ARMBRUSTER Loup
KHODJAOUI Elias

IUT Nancy-Charlemagne
3ème année

Groupe : RA-IL-1

https://github.com/Kuuro57/VR-4D_Projet_Tutore

Projet Tutoré :
S9 : Création d'une application en VR
Compte rendu semestre 1

Tuteur du projet : DUPONT Laurent (laurent.dupont@loria.fr)
Responsable des projets : BINET Julien (julien.binet@univ-lorraine.fr)

Table des matières

Table des matières 2
Introduction 3
Cahier des charges et objectifs 4
Analyse initiale 5
Rétrospectives sur les itérations 7
Itération 1 : Gestion des formes 7
Itération 2 : De la 3D à la 4D 10
Journée portes ouvertes 12
Itération 3 : De la 4D à la VR 13
Perspectives : fin du projet 15
Conclusion du semestre 15
Annexes 16

Introduction

Ce document a pour but de faire le point sur l'avancement du projet depuis ses débuts jusqu'à la fin du premier semestre, comptant au total 3 itérations, chacune d'une durée d'une semaine. Nous allons repartir du cahier des charges et des objectifs fixés lors de l'analyse initiale du projet pour comparer ce qui était prévu avec ce qui a réellement été fait au cours de ces trois itérations.

Le sujet de notre projet est la réalisation d'une application en VR (réalité virtuelle) capable de rendre possible la visualisation d'une forme géométrique quelconque en 4D dans un espace en 3D. Concrètement, l'objectif de ce projet est de rendre la visualisation et la compréhension d'un objet en 4D plus intuitive.
Pour ce faire, nous avons à disposition un casque VR (Meta Quest 3) pour visualiser les objets géométriques dans un espace 3D. Pour le développement de l'application en elle-même, nous utilisons le langage JavaScript, et plus précisément la libraire Babylon.js. Cette bibliothèque est un moteur de rendu 3D pour les navigateurs web, et qui permet aussi nativement de créer et modéliser un environnement 3D compatible avec la VR.
Ce sujet repose sur deux piliers : la maîtrise technique de Babylon.js pour la réalité virtuelle et l'application de concepts mathématiques, tels que les matrices de rotation et les calculs de vecteurs.

Cahier des charges et objectifs

Pour mener à bien la réalisation de ce projet, nous avons eu quelques contraintes données par notre tuteur de projet.
Comme dit précédemment, nous devons utiliser la libraire Babylon.js pour avoir un moteur de rendu 3D déjà utilisable, sans quoi nous aurions dû le refaire de toute pièce, ce qui n'est pas chose facile compte tenu du temps donné pour réaliser l'application. Une deuxième contrainte est l'utilisation du casque VR pour visualiser la forme 4D, ce qui nous oblige à nous renseigner davantage sur les fonctionnalités de Babylon.js pour faire tourner notre application sur un casque VR.
L'objectif minimal à atteindre pour la fin du projet est la visualisation d'une forme 4D au travers du casque VR, ainsi que la possibilité d'interagir avec la forme 4D dans l'espace 3D (rotations, déplacement, agrandissement et rétrécissement).

Pour avancer efficacement, nous avons divisé le projet en plusieurs objectifs que nous avons catégorisé :

Objectifs critiques	**Objectifs complémentaires**
Projection d'une forme 4D (en 3D) dans la scène Babylon.js	Modéliser une scène Babylon.js en 3D
Rotation / Translation d'une forme 4D	Rotation d'un objet 3D à l'aide de matrices de rotations
Affichage d'une interface des différents plans de la figure 4D	Translation d'un objet 3D
Affichage d'une forme 4D en VR	Gestion des déplacements en VR (manettes / caméra)
	Affichage d'une forme 3D en VR

Ces objectifs sont là pour avoir un fil conducteur que nous pouvons suivre, et qui nous permettent aussi de nous faire une idée du bon avancement ou non du projet par rapport au délai fixé.

Analyse initiale

Avant de démarrer le projet, nous avons dû préparer le terrain en faisant une analyse initiale des contraintes, objectifs, étudier l'existant et explorer les défis techniques qui nous attendaient (surtout pour la partie mathématique).
Dans un premier temps, nous avons lu la documentation de Babylon.js pour essayer de comprendre son fonctionnement, puis nous avons développé des environnement 3D simples pour nous familiariser avec le moteur de rendu 3D. En parallèle, nous avons fait des recherches sur internet pour voir ce qui avait été fait avec Babylon.js, et plus précisément ce qui existait par rapport à la 4D. Nous n'avons trouvé qu'une seule vidéo YouTube montrant un environnement 3D au travers d'un casque VR affichant une forme en 4D. La seule différence avec notre projet était que l'application tournait grâce à Unity (un moteur de rendu 3D généralement utilisé pour le jeu vidéo) et non pas avec Babylon.js.

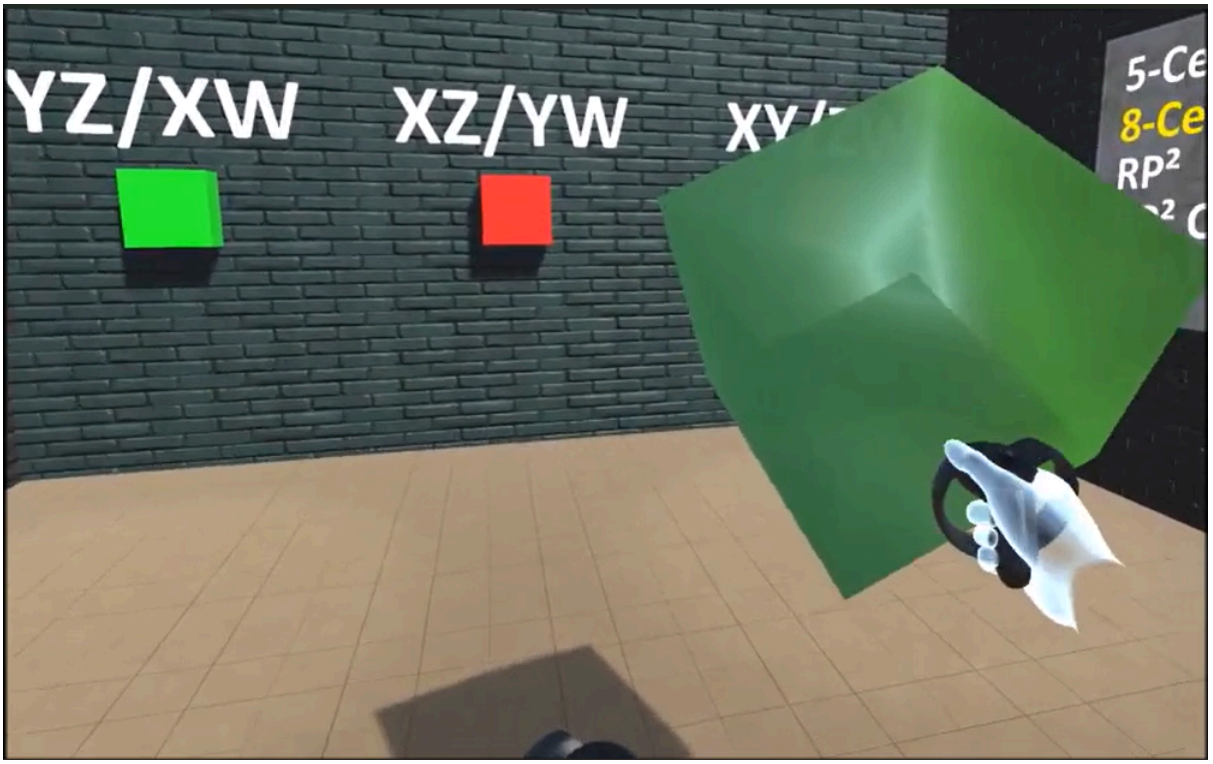

Figure n°1 - Visualisation d'une forme 4D sous Unity

Dans un second temps, nous nous sommes penchés sur l'étude technique sur l'existence de solutions. Cette partie nous a été d'une grande aide pour la suite, car c'est grâce à ces recherches que nous avons pu avancer rapidement sur le développement pure de l'application. Nous avons recherché sur beaucoup de sites comment fonctionnait la quatrième dimension, mais aussi comment était-il possible de visualiser une forme en 4D dans un espace 3D. Nous avons finalement trouvé le concept de plan projectif, qui permet de passer d'une dimension N à une dimension N-1 (exactement ce qu'il nous faut pour passer de la 4D à la 3D).

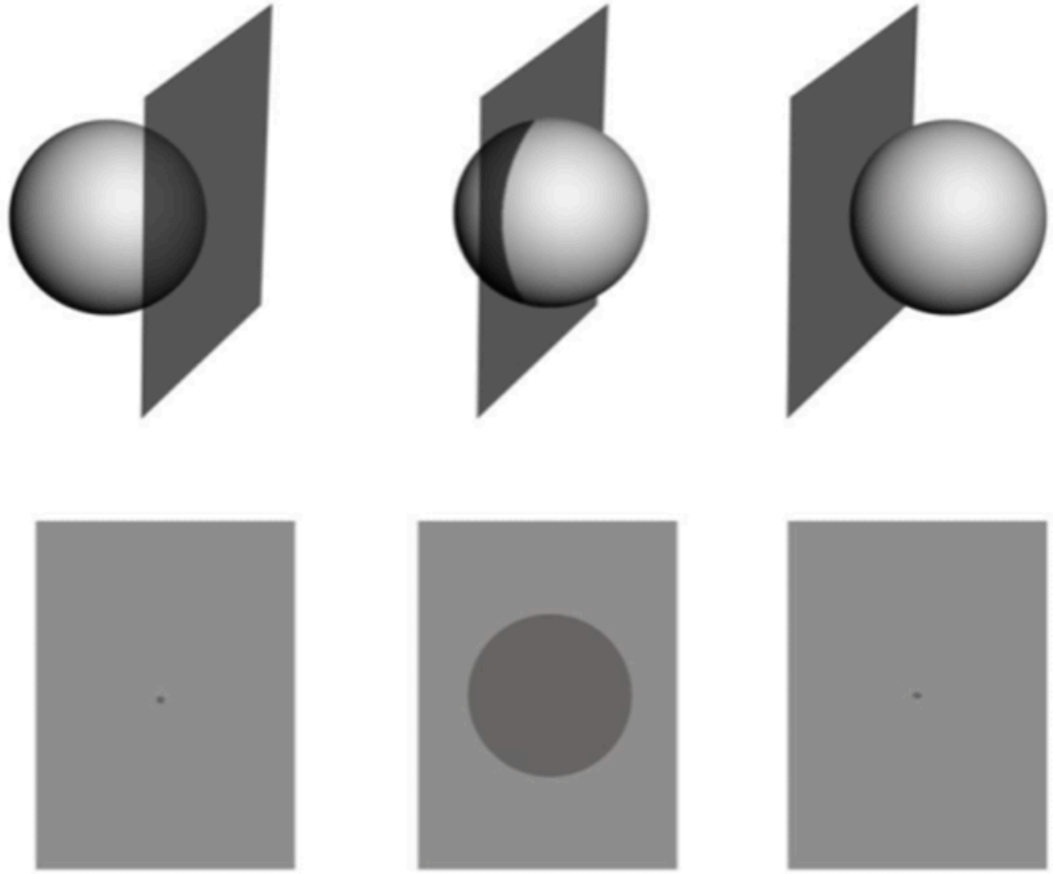

Figure n°2 - Exemple de plan projectif (3D → 2D)

Une fois celà fait, il fallait que l'on puisse effectuer des opérations sur notre forme 4D. Après de plus amples recherches, nous avons trouvé les matrices de rotations pour faire pivoter une forme 4D selon un plan. Pour ce qui est de la translation d'une forme 4D, c'est la même chose qu'en 3D.

Pour finir, nous avons étudié plus en profondeur la façon dont fonctionne Babylon.js pour représenter des objets dans son espace 3D. En effet, Babylon.js utilise des Vector3 (des vecteurs avec 3 coordonnées) pour positionner un élément dans l'environnement. Or nos formes en 4D ont 4 coordonnées (x, y, z, w). Pour représenter nos objets en 4D dans Babylon.js, nous avons imaginé créer des objets représentant nos formes 4D que nous passions en 3D (en utilisant des Vector3) avec l'utilisation d'un plan projectif, pour pouvoir afficher notre forme dans l'espace 3D.

Rétrospectives sur les itérations

Itération 1 : Gestion des formes

Pour la première itération, avec nos connaissances en javascript et notre analyse de la librairie Babylon.js, nous avons commencé par instancier des formes (un cube pour commencer).

Après avoir compris les bases, nous avons décidé de définir des classes pour plusieurs raisons :

- Générer facilement une forme sans avoir à toucher directement à la 3D (instancier les points est suffisant)
- Pour un travail collaboratif

En effet, un système de classes nous a permis de diviser une fonctionnalité (cube : développée par Elias) en plusieurs sous-classes pour définir une figure géométrique:

- Sommets (développé par Loup)
- Arêtes (développé par Noah)
- Faces (développé par Loup)

Une fois un cube affiché facilement en quelque lignes de codes (à l'aide d'une méthode de génération), notre but suivant a été d'interagir avec la forme à l'aide d'une interface :

- Translation (développée par Loup)
- Homothétie (développée par Elias)
- Rotation 3D (développée par Noah)

De plus, nous souhaitions ajouter différentes méthodes pour faciliter la suite du projet :

- Nous avons changé l'architecture, passant d'une classe Cube a une classe plus générique Forme capable d'afficher une forme en 3D comme en 4D (développée par Noah)

- Nous avons codé une méthode pour générer une figure générique en donnant la liste de ses sommets et arêtes (donnés dans un fichier JSON) (développée par Elias)

- Nous avons affiché les différentes projections 2D du cube dans les coins de l'écran (développée par Loup)

- Nous avons séparé la création de figures en différentes méthodes de la forme "load…From…()" pour générer chaque figure. Comme chaque figure a tous ses paramètres en commun, son affichage et les transformations qui lui sont appliquées restent les mêmes peu importe la figure (à l'exception de la rotation 4D) (développée par Loup, Noah et Elias)

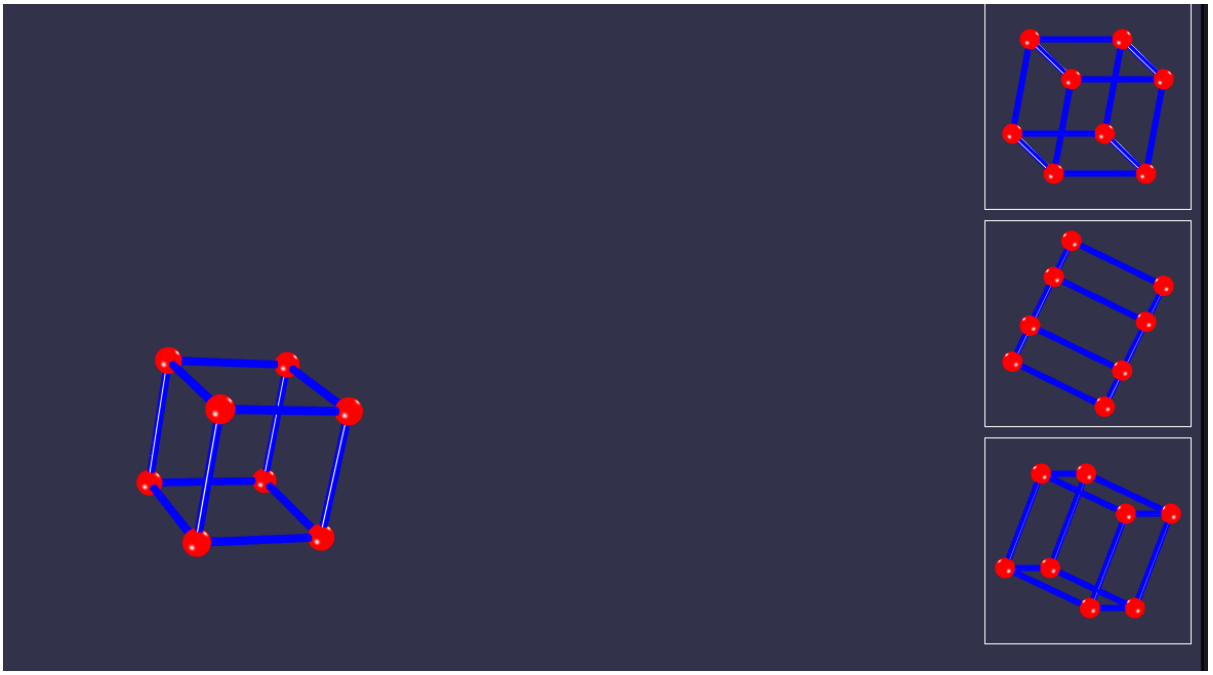

Figure n°3 - Affichage d’un cube et de ses projections 2D (vue fil de fer)

Cependant, avec le recul de 3 itérations, nous nous sommes rendus compte que l’utilisation de fichiers JSON est obsolète, et nous hésitons à l’améliorer ou à l’abandonner complètement.

Cette première itération a été en grande partie de la découverte, mais nous avons très bien distribué le travail, nous permettant de compléter toutes ces fonctionnalités pour la première itération. Nous avons donc pu présenter une scène BabylonJS avec un cube où toutes les transformations étaient codées correctement.
De plus, nous étions capables à la fin de l’itération d’afficher un hypercube (4D), de le déplacer, mais pas de le faire tourner.

Itération 2 : De la 3D à la 4D

Lors de la deuxième itération, comme toutes les opérations primaires étaient codées, nous avons pu nous concentrer uniquement sur la réflexion et le développement en 4D. Nous avons beaucoup discuté de comment implémenter les différentes figures, de comment elles se représentent et comment afficher leurs projections sur l'écran. En effet, la grande différence avec les formes 3D est que l'on ne peut pas afficher la figure en elle-même, mais seulement afficher une projection 3D d'une de ces formes 4D.

Après nous être entretenus avec notre tuteur Laurent Dupont, nous avons choisi "d'aplatir" une des 4 dimensions pour afficher les 3 dimensions restantes. En effet, en fonction de la projection, nous n'affichons pas les 3 mêmes dimensions. rendant chaque projection différente.

En termes d'organisation, cette itération fut plus compliquée, étant donné que l'on modifiait les mêmes fichiers pour les mêmes fonctionnalités, nous forçant à résoudre plusieurs merge git, nous ralentissant.

Cependant, chaque membre cherchait à faire quelque chose d'utile :

- Ajout de faces à l'hypercube et affichage des projections 3D (développé par Loup)
- Réglage de bugs et déplacement de l'hypercube (développé par Noah)
- Ajout de la rotation 4D et codage d'une sphère et hypersphère (développé par Elias)

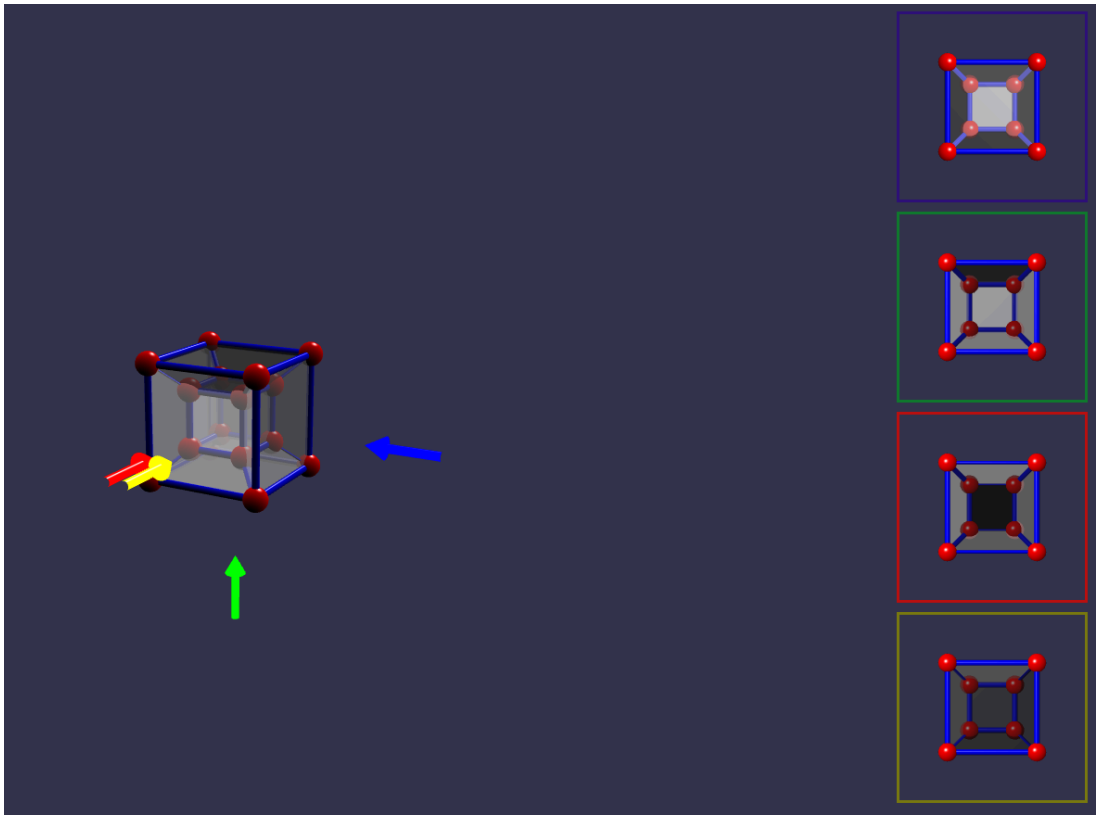

Figure n°4 - Affichage d’un hypercube et de ses différentes projections 3D

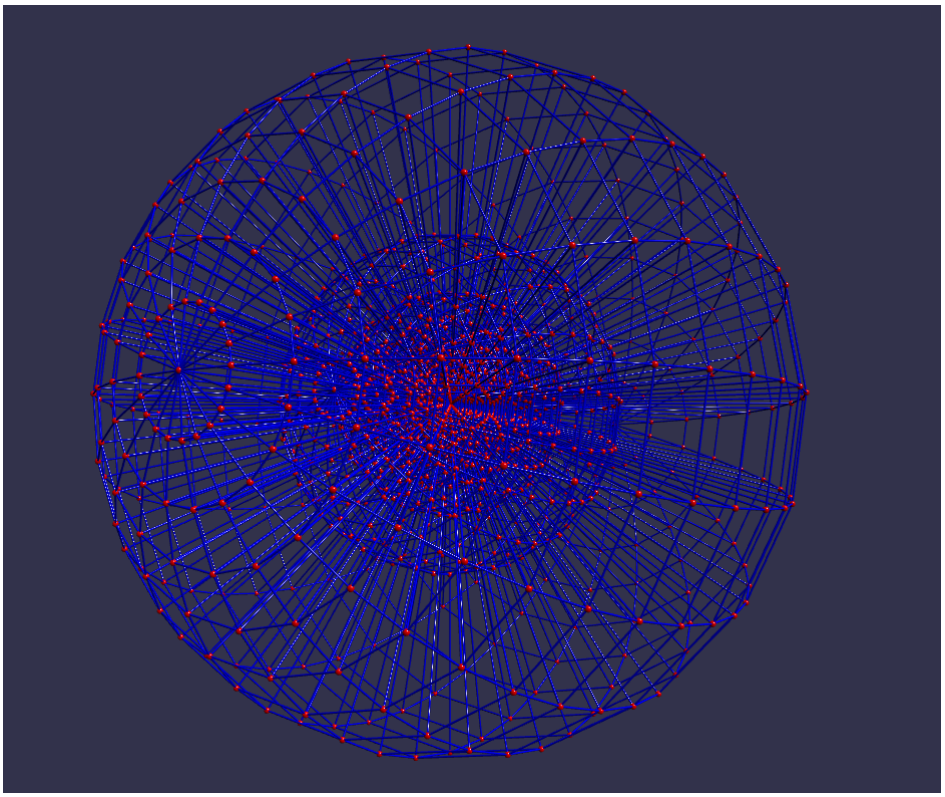

Figure n°5 - Affichage d’une hypersphère de près de 8000 points (20 sommets par segment)

Dans cette itération, nous nous sommes rendus compte que notre interface, bien que fonctionnelle, était peu utile pour le passage imminent à la VR. De plus, les contrôles s'étendaient sur une centaine de lignes de code, nous poussant à dupliquer les pages internet pour la 3D et la 4D (hypercube/hypersphère), plutôt que de réécrire tout notre code.

Il nous est apparu qu'il était nécessaire d'ici l'itération suivante d'avoir une page d'affichage propre, avec des contrôles plus simples (touches du clavier) facilitant le passage à la VR.

Nous sommes parvenus, malgré ces problèmes, à délivrer pour la fin de l'itération la 4D fonctionnelle (à l'exception de bugs liés aux rotations et translations en 4D), avec comme objectif suivant de commencer à passer en VR, tout en perfectionnant la 4D.

Journée portes ouvertes

Entre l'itération 2 et 3, nous avons eu le plaisir de participer à la journée portes ouvertes de l'IUT Nancy Charlemagne.

Dans ce cadre, nous avons pu présenter notre projet à des visiteurs et d'autres étudiants de l'IUT, recueillant par la même occasion des retours utilisateurs pour orienter la suite de notre projet.

En effet, nous avons remarqué notre besoin constant de recharger la page pour réinitialiser l'état de la figure. Une piste d'amélioration serait donc d'ajouter un bouton reset sur la page.

D'autre part, notre tuteur nous a proposé de colorer les faces différemment pour aider les utilisateurs à comprendre ce qui se passe lors des rotations.

Itération 3 : De la 4D à la VR

Pour cette troisième itération nous avons pu passer à une autre grosse partie de notre projet qui est le passage de la notre application web à une application en réalité virtuelle.

Pour appréhender le passage à la VR nous avons commencé par un refactor de notre code et par un retrait du menu d'interaction (fait par Noah) qu'il y avait auparavant pour relier chaque interaction à une touche du clavier puisqu'un des objectifs du passage à la VR est de pouvoir utiliser nos mains pour interagir avec la forme 4D.

En ce qui concerne la VR, nous avons commencé par chercher du code dans la documentation pour voir comment l'implémenter dans notre code. Nous avons été d'ajouter au projet un nouvel index auquel le casque VR peut se connecter (uniquement en https avec besoin de génération de certificat), lancer une "expérience VR" (mode déplacement VR) où l'on affiche un cube, soit une forme que l'on instancie nous même. (développé par Elias, Noah et Loup)
Nous avons commencé à relier les boutons des manettes à des contrôles temporairement, mais ils ne sont pas parfaits et présentent encore quelques problèmes notamment sur les rotations. De plus, nous n'avons pas encore créé des contrôles pour la VR sans manettes.

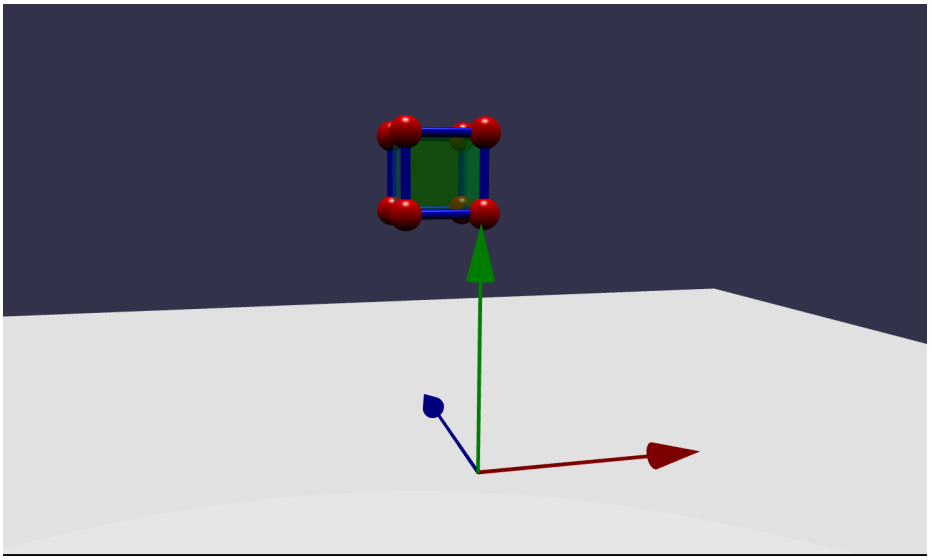

Figure n°6 - Vue VR d'une scène exemple, avec un cube pouvant être mû

Parallèlement, nous avons bien ajouté une coloration des faces (développée par Loup) comme recommandé par notre tuteur pour avoir une meilleure visibilité et compréhension de ce qu'il se passe lorsque l'utilisateur interagit avec la forme 4D.
Nous avons aussi ajouté un pentachore (ajouté par Elias) qui est un tétraèdre (pyramide) en 4D, qui est une des formes géométriques qui contient le moins de faces, d'arêtes et de sommets, considérée comme la figure 4D la plus "simple".

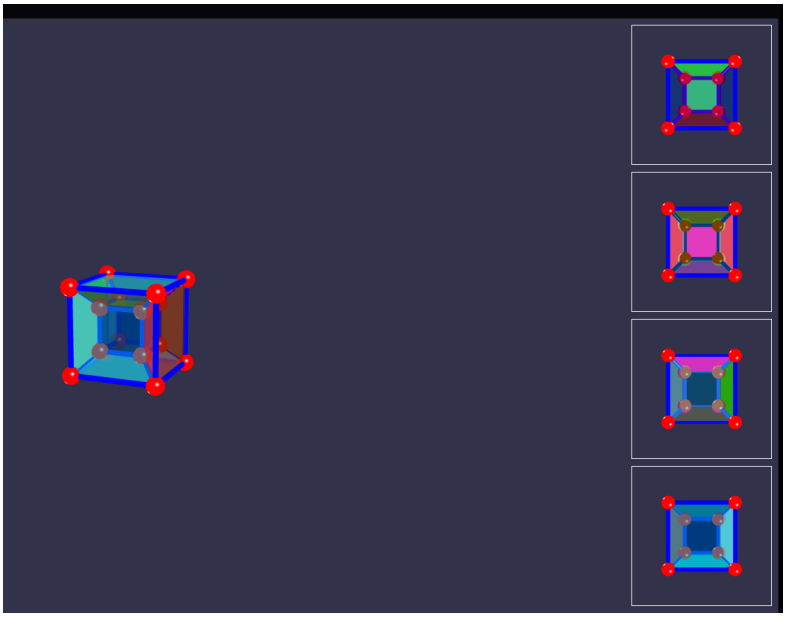

Figure n°7 - Hypercube final avec faces colorées

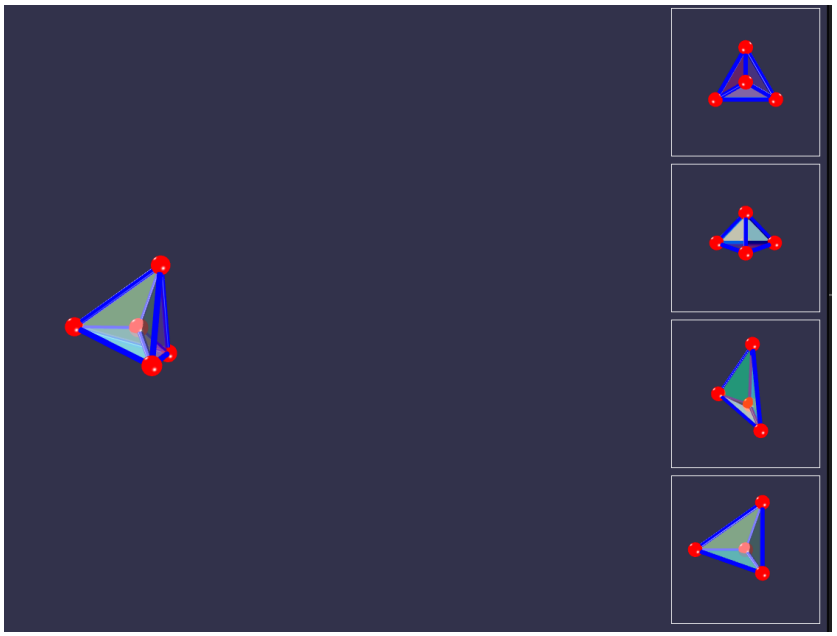

Figure n°8 - Affichage d'un pentachore

Perspectives : fin du projet

Nous sommes à la moitié de notre projet tutoré et bien que nous ayons déjà avancé nous avons encore des idées claires des ajouts de fonctionnalités qu'ils soient critiques ou non que nous allons développer.

Lors de la quatrième itération nous allons finaliser le passage à la VR avec toutes les intéractions puisqu'il manque certains liens comme les translations.
Nous allons aussi rajouter un bouton pour réinitialiser la figure puisqu'il est plus simple de comprendre les modifications appliquées à la forme 4D avec la forme de base, cet ajout aurait dû être ajouté lors de la troisième itération mais nous n'avons pas eu le temps.

L'un des axes de progression sera bien sûr l'optimisation des interactions avec l'hypersphère puisque le nombre de calculs effectués est très volumineux et rend très saccadé chaque mouvement.

Lors d'une réunion avec notre tuteur nous avons aussi évoqué une piste d'exploration en rapport avec les voxels. Ce sont des équivalents de pixels dans la troisième dimension. L'utilisation des voxels dans notre projet permettrait d'avoir des représentations différentes de nos formes 4D ainsi que de simplifier les calculs fait avec par les interactions de l'utilisateur à la forme géométrique

Conclusion du semestre

Lors de ce premier semestres, nous avons bien progressé dans ce projet tutoré visant à nous faire découvrir des technologies que nous avons peu voir non utilisées.

En effet, lors de ces trois itérations nous sommes passés de la découverte de la librairie Babylon.js pour avoir une forme dans la troisième dimension avec laquelle

nous pouvons interagir. Pour la transformer dans l'itération suivant à une forme dans la quatrième dimension. Pour la placer dans l'itération suivante dans un espace en VR. Chaque itération nous a alors permis de développer des fonctionnalités critiques.

Pendant ces trois semaines nous avons pu mettre en application nos cours en javascript vu en deuxième année ainsi que pousser les cours de maths que nous avons vu peu de temps avant le début des projets. Tout en pratiquant les pratiques de qualité de développement que nous avons vu dans le cours éponyme comme la documentation, l'utilisation de git ainsi que les principes SOLID. Ce projet nous entraîne aussi à travailler en équipe, la collaboration, l'entraide, et la répartition des tâches. Et lors de cette troisième semaine nous avons pu utiliser le casque de réalité virtuelle "Meta quest 3" qui est une technologie que nous n'aurons sûrement jamais utilisé hors de ce projet ce qui est une bonne découverte.

Annexes

Documentation Babylon.js :
https://doc.babylonjs.com/

Exemples d'utilisation de Babylon.js :
https://playground.babylonjs.com/#WJXQP0
https://playground.babylonjs.com/#TJIGQ1#3
https://playground.babylonjs.com/#ja1nd3#942

Lien de la vidéo YouTube de l'application utilisant Unity :
https://youtu.be/1Zamk9ZOrjk?si=dRbxmKKnmsOTaYjZ

Définition d'un voxel :
https://fr.wikipedia.org/wiki/Voxel